claude-windows / ce5ec24b-cf40-49bb-afb0-3a41f4cc9934

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ce5ec24b-cf40-49bb-afb0-3a41f4cc9934\subagents\workflows\wf_4b294c39-6a7\agent-acf7cf72f34283ef9.jsonl`
- SHA-256: `616334a94e30baa1efa650aa821aeb1102752f55363f487d40e4d5925c6abf77`
- Source modified: `2026-06-22T18:51:16+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_4b294c39-6a7`
- session_id: `ce5ec24b-cf40-49bb-afb0-3a41f4cc9934`

Transcript

1. user (2026-06-22T18:49:26.603Z)

Engine repo at C:/Users/avidu/Projects/bhi-computeval, backend/main.py. Verify the EXACT reel-creation API flow + endpoints a client uses: GET /api/capabilities, POST /api/process (NOW ASYNC: 202 + job_id, poll /api/jobs/{id}), GET /api/jobs/{id}, POST /api/query, POST /api/compile (NOW ASYNC too, #329 ? 202 + job_id, poll), GET /api/highlights/{video_id}/{filename}, chunked upload (/api/upload/*). For each: method, path, request body fields, response shape, and how they chain (upload/gs:// ? process ? query ? compile ? download). Note the available segmenters (segmenter_registry / default) + the job result['compute'] provenance block. Report only VERIFIED facts with file:line citations ? this doc must reflect SHIPPED state, never aspiration. If something is NOT implemented, say so.

2. assistant (2026-06-22T18:49:29.221Z)

I'll verify the reel-creation API flow in the engine repo. Let me start by pulling the repo fresh and locating the relevant files.

3. user (2026-06-22T18:49:33.096Z)

Your configuration specifies to merge with the ref 'refs/heads/feat/compute-provenance'
from the remote, but no such ref was fetched.
---BRANCH---
feat/compute-provenance
---HEAD---
ee6fba8 feat(serving): forward skip_ai_handoff to the cloud job so the picker's cloud path yields rallies
4b6a560 feat(serving): compute-path provenance ? prove which backend ran the DETECT
8bcf6ce feat(api): make /api/compile async (202 + poll) to kill the slow-stitch 524 (#329) (#339)

4. user (2026-06-22T18:49:33.165Z)

C:/Users/avidu/Projects/bhi-computeval/backend\main.py

5. assistant (2026-06-22T18:49:36.528Z)

The repo is on branch `feat/compute-provenance`. The git config wants to track a remote ref that wasn't fetched, but I have the local checkout. Let me note that I'm reading what's local (which includes the unmerged compute-provenance work) and read main.py.

6. user (2026-06-22T18:49:40.730Z)

---master vs HEAD---
d92cc10 feat(serving): make the cloud compute-picker path work + provable (provenance + skip_ai_handoff forward) (#341)

```
8bcf6ce feat(api): make /api/compile async (202 + poll) to kill the slow-stitch 524 (#329)
(#339)
819e395 feat(serving): M12 ? gdrive-api:// OAuth/API StorageBackend (D14 Northstar) (#338)
---wc main.py---
1390 backend/main.py
```

7. user (2026-06-22T18:49:41.210Z)

```
<system-reminder>[Truncated: PARTIAL view ? showing lines 1-1038 of 1391 total (28462 tokens,
cap 25000). Call Read with offset=1039 limit=1038 for the next page, or Grep to find a specific
section. Do NOT answer from this page alone if the answer may be further in the file.]</system-
reminder>

1       import os
2       import sys
3       import argparse
4       import csv
5       import json
6       import logging
7       import math
8       import shutil
9       import socket
10      import tempfile
11      import threading
12      import uuid
13      import webbrowser
14      from importlib.metadata import PackageNotFoundError, version as _pkg_version
15      from urllib.parse import quote
16      import uvicorn
17      from dotenv import load_dotenv
18
19      # PACKAGING_PLAN.md P0a: prefer an app-home .env ONLY when RALLY_APP_HOME is set (the
packaged
20      # launcher pins it); otherwise env_file_to_load() returns None and load_dotenv(None)
keeps the
21      # exact historical frame-relative walk-up ? byte-identical to the pre-P0a bare
load_dotenv().
22      # stdlib-only import ? keeps this pre-pydantic line light.
23      from backend.utils.app_home import env_file_to_load, resolve_output_dir
24
25      load_dotenv(env_file_to_load())
26
27      # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print
for direct user-facing messages).
28      logging.basicConfig(
29          level=logging.INFO,
30          format='%(asctime)s [%(levelname)s] %(name)s: %(message)s',
31          datefmt='%H:%M:%S'
32      )
33      logger = logging.getLogger(__name__)
34      from fastapi import FastAPI, HTTPException, Request
35      from fastapi.middleware.cors import CORSMiddleware
36      from starlette.middleware.trustedhost import TrustedHostMiddleware
37      from pydantic import BaseModel
38      from typing import List, Dict, Any, Optional, Tuple
39
40      from fastapi.staticfiles import StaticFiles
41      from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
42      from backend.pipeline.validators.base import registry
43      from backend.infrastructure.database import Database
44      from backend.pipeline.segmenters.base import segmenter_registry
45      from backend.pipeline.stitcher.compiler import VideoCompiler
46      from backend.pipeline.stitcher.watermark_i18n import localize_watermark
47      from backend.utils.hashing import compute_video_id  # canonical file-identity hashing
48      from backend.utils.paths import resolve_under_root, safe_output_filename,
```

```
validate_input_path
49      from backend.utils.redact import redact_secrets  # P0b: mask secret-shaped fields in
/api/config
50      from backend.utils.single_instance import acquire_lock  # P0e: one instance per database
51      from backend.utils.ffmpeg import ffmpeg_bin, ffprobe_bin  # P2a: single ffmpeg/ffprobe
resolver
52      from backend.config import (  # new typed config
53          load_config as load_app_config,
54          write_light_default_config,  # P2b: batteries-included first-run config seeding
55          AppConfig,
56          StitchingConfig,
57          enforce_startup_checks,
58          StartupCheckError,
59      )
60      from backend.results import Failure  # standardized error/result contract
61      from backend.reporting import emit_indexer_report
62      from backend import storage  # M2: URI-aware input acquisition (local = identity
passthrough)
63      from backend import billing  # M8: per-job cost ledger (USD internal / INR display)
64      from backend.api import auth as edge_auth  # M9: Cloudflare Access edge-auth (default-
OFF)
65
66      app = FastAPI(title="Sports Highlight Indexer API")
67
68
69      def _cors_origins_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:
70          """M9: CORS allow-origins from the ``RALLY_CORS_ALLOW_ORIGINS`` env var (comma-
separated),
71          default ``["*"]``. Read from the ENV (not a cwd ``config.json``) so importing
``backend.main``
72          does zero filesystem I/O ? a hosted/multi-tenant deploy sets this to lock CORS to
its origin."""
73          src = os.environ if env is None else env
74          raw = (src.get("RALLY_CORS_ALLOW_ORIGINS", "") or "").strip()
75          origins = [o.strip() for o in raw.split(",") if o.strip()]
76          return origins or ["*"]
77
78
79      # CORS for the web UI. allow_origins='*' with allow_credentials=True is rejected by
browsers per the
80      # fetch spec + is unsafe to carry into multi-tenant; this tool uses no cookies, so
credentials stay
81      # off. The origin list is configurable via the env var above (default ["*"]) so a hosted
deploy can
82      # lock it to its origin; the middleware is fixed at startup.
83      app.add_middleware(
84          CORSMiddleware,
85          allow_origins=_cors_origins_from_env(),
86          allow_credentials=False,
87          allow_methods=["*"],
88          allow_headers=["*"],
89      )
90
91
92      def _allowed_hosts_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:
93          """P0c: Host-header allowlist ? a DNS-rebinding guard for the localhost appliance. A
malicious
94          web page can point a domain it controls at 127.0.0.1 and script requests to the
local server;
95          the browser sends ``Host: attacker.com``, which this rejects (the server only
answers loopback
96          Host headers by default). ``RALLY_ALLOWED_HOSTS`` (comma-separated) overrides; the
default keeps
97          Starlette's TestClient host ``testserver`` so the in-process suite isn't broken.
Read from the
```

```
98          ENV (not config.json) so importing ``backend.main`` does zero filesystem I/O; a
routable/edge
99          deploy sets its public host (or ``*`` when fronted by an auth gate that validates
the host)."""
100         src = os.environ if env is None else env
101         raw = (src.get("RALLY_ALLOWED_HOSTS", "") or "").strip()
102         hosts = [h.strip() for h in raw.split(",") if h.strip()]
103         # IPv4 loopback (the default bind) + the TestClient host. NB: Starlette's
TrustedHostMiddleware
104         # matches on host.split(":")[0], which mangles a bracketed IPv6 Host ("[::1]:port"),
so IPv6
105         # loopback is NOT supported by this guard ? access the appliance via
127.0.0.1/localhost.
106         return hosts or ["localhost", "127.0.0.1", "testserver"]
107
108
109     # DNS-rebinding guard. Bound to localhost by default, so only loopback Host headers are
honoured
110     # (a remote page can't read responses cross-origin AND its Host header is rejected
here). www_redirect
111     # off so an unexpected ``www.`` host is rejected rather than 307-redirected.
112     app.add_middleware(
113         TrustedHostMiddleware,
114         allowed_hosts=_allowed_hosts_from_env(),
115         www_redirect=False,
116     )
117
118
119     def _resolve_bind_host(config: Optional[AppConfig] = None, cli_host: Optional[str] =
None) -> str:
120         """P0c: the server bind address. Precedence: ``--host`` > ``RALLY_BIND_HOST`` env >
``0.0.0.0``
121         when an edge-auth profile is active (``security.edge_auth_enabled`` ? i.e. there IS
an auth gate
122         in front) > ``127.0.0.1`` (default: localhost-only, today's behaviour). Binding a
routable
123         interface with no auth gate is unsafe, hence the localhost default; widen it
consciously."""
124         if cli_host and cli_host.strip():
125             return cli_host.strip()
126         env = os.environ.get("RALLY_BIND_HOST")
127         if env and env.strip():
128             return env.strip()
129         sec = getattr(config, "security", None)
130         if getattr(sec, "edge_auth_enabled", False):
131             return "0.0.0.0"  # behind a Cloudflare-Access-style edge gate (M9) ? bind all
interfaces
132         return "127.0.0.1"
133
134
135     # --- P2: the friendly `indexer --app` launcher (cross-platform, no native component ?
O1) ---
136
137     def _port_is_free(host: str, port: int) -> bool:
138         """True if (host, port) can be bound right now (best-effort; a TOCTOU race is
acceptable ?
139         uvicorn surfaces a real bind failure loudly)."""
140         with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
141             try:
142                 s.bind((host, port))
143                 return True
144             except OSError:
145                 return False
146
147
```

```
148     def _find_free_port(host: str) -> int:
149         """An OS-assigned free port on host."""
150         with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
151             s.bind((host, 0))
152             return int(s.getsockname()[1])
153 
154 
155     def _resolve_app_port(host: str, port: int) -> int:
156         """For --app: keep `port` if free, else fall back to a free one so a busy 8000 never
blocks the app."""
157         return port if _port_is_free(host, port) else _find_free_port(host)
158 
159 
160     def _open_browser_when_ready(url: str, health_url: str, timeout: float = 60.0) -> bool:
161         """Poll `health_url` until the server is up, then open `url` in the user's default
browser.
162         Returns True if a browser open was attempted. No-op (returns False) when
RALLY_NO_BROWSER is set
163         (tests / headless) or the server never becomes healthy (don't open a dead page).
Never raises."""
164         if os.environ.get("RALLY_NO_BROWSER"):
165             return False
166         import time as _time
167         import urllib.request
168         deadline = _time.monotonic() + timeout
169         while _time.monotonic() < deadline:
170             try:
171                 with urllib.request.urlopen(health_url, timeout=2) as r:
172                     if getattr(r, "status", 200) == 200:
173                         break
174             except Exception:
175                 _time.sleep(0.3)
176         else:
177             return False
178         try:
179             webbrowser.open(url)
180         except Exception:
181             pass
182         return True
183 
184 
185     # Serves static frontend files directly (now under api/ per approved structure).
186     # PACKAGING_PLAN.md P0a: resolve PACKAGE-relative (not CWD-relative) so launching from
an
187     # arbitrary directory finds the shipped assets instead of creating a stray
./backend/api/static.
188     # (The legacy in-engine dashboard; the real SPA gets bundled here in P1a.)
189     _STATIC_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "api", "static")
190     try:
191         os.makedirs(_STATIC_DIR, exist_ok=True)
192     except OSError:
193         # The dir ships with the package (a no-op create); guard against a read-only install
194         # tree (system-installed Python) since exist_ok suppresses FileExistsError, not
PermissionError.
195         pass
196     app.mount("/static", StaticFiles(directory=_STATIC_DIR), name="static")
197 
198     # P1b: serve the bundled KhelSutra SPA single-origin. `backend/ui/` is the committed
snapshot
199     # (produced by scripts/bundle_ui.sh); fall back to the legacy in-engine dashboard only
if it's
200     # absent (a source checkout with no vendored UI). The SPA calls same-origin `/api`, so
no SPA change
201     # is needed; it is query-param driven (?video=?), so serving index.html at `/` + /assets
is enough
```

```
    # (no client-side path routing ? no history-fallback catch-all, which would also be a
route-order hazard).
    _BUNDLED_UI_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "ui")
    _UI_DIR = _BUNDLED_UI_DIR if os.path.isfile(os.path.join(_BUNDLED_UI_DIR, "index.html"))
else _STATIC_DIR
    _UI_ASSETS_DIR = os.path.join(_UI_DIR, "assets")
    if os.path.isdir(_UI_ASSETS_DIR):
        app.mount("/assets", StaticFiles(directory=_UI_ASSETS_DIR), name="assets")


    @app.get("/", response_class=HTMLResponse)
    def serve_index():
        index_path = os.path.join(_UI_DIR, "index.html")
        if os.path.exists(index_path):
            with open(index_path, "r", encoding="utf-8") as f:
                return f.read()
        return "<h3>Sports Highlight Indexer Server is Running. (No bundled UI found.)</h3>"


    def _bundled_ui_version() -> Optional[Dict[str, Any]]:
        """Provenance of the vendored SPA (backend/ui/ui_version.json), or None if no UI is
bundled.
        Includes `engine_api_version` ? the API_VERSION the snapshot was built against (skew
guard)."""
        path = os.path.join(_BUNDLED_UI_DIR, "ui_version.json")
        try:
            with open(path, "r", encoding="utf-8") as f:
                return json.load(f)
        except (OSError, ValueError):
            return None

    # Global variables initialized at startup
    db_instance: Optional[Database] = None
    global_config: Optional[AppConfig] = None  # now typed (Pydantic model)
    # P0e: the single-instance lock handle. Kept module-global for the process lifetime ? if
it were
    # GC'd/closed the OS would release the lock. None until main() acquires it (server/CLI
init).
    _instance_lock: Optional[Any] = None

    # Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3)
    # now lives in backend.api.job_worker. Re-exported here so the names stay addressable on
    # `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
    # `_run_claimed_job`, `_MAX_DRAIN_WAKE_SEC`, ...). The worker resolves these seams
THROUGH
    # this module object at call-time, so `monkeypatch.setattr(backend.main, ...)` still
    # intercepts the inter-function calls (a parked job arms the PATCHED `_arm_wake_timer`,
    # so no real daemon timer leaks in tests). Behavior is unchanged by the extraction.
    from backend.api.job_worker import (  # noqa: F401  (re-exports: names stay addressable
on backend.main)
        JobWaiting,
        _arm_wake_timer,
        _drain_due_jobs,
        _get_executor,
        _job_to_request,
        _MAX_DRAIN_WAKE_SEC,
        _maybe_record_job_cost,
        _run_claimed_job,
        _schedule_next_drain_if_waiting,
    )

    # Legacy thin wrapper for any remaining direct calls during transition.
    # New code should import load_app_config / AppConfig from backend.config directly.
    def load_config(config_path: str = "config.json") -> Dict[str, Any]:
```

```
258         cfg = load_app_config(config_path)
259         return cfg.model_dump(mode="json")
260
261     # --- API Models ---
262     class ProcessRequest(BaseModel):
263         video_path: str
264         player_pool: Optional[List[str]] = None
265         max_frames: Optional[int] = None
266         segmenter: Optional[str] = None
267         # The UI locale the SPA was rendered in (e.g. "kn", "hi-IN"). Recorded on the job
for PMF
268         # analytics (count_jobs_by_locale). Optional ? NULL when unrecorded (CLI / older
clients).
269         locale: Optional[str] = None
270         # SPA compute-picker (item 3): "local" forces in-process even on a cloud-configured
server;
271         # "cloud" forces a Cloud Run dispatch (requires deployment.cloud_available). None ?
server
272         # default (deployment.compute_target). Only honoured for these two values; see
_maybe_dispatch_remote.
273         compute: Optional[str] = None
274
275     class QueryRequest(BaseModel):
276         video_id: str
277         server: Optional[str] = None
278         receiver: Optional[str] = None
279         duration_min: Optional[float] = None
280         event_type: Optional[str] = None
281
282     class CompileRequest(BaseModel):
283         video_id: str
284         segment_ids: List[int]
285         output_filename: str
286         # The UI locale the SPA was rendered in (e.g. "kn", "hi-IN"). Localizes the reel
watermark
287         # (text + script font). Optional ? absent/unknown keeps the configured default (en
behavior).
288         locale: Optional[str] = None
289
290     def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:
291         """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
292
293         On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
294         fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
and
295         keep the prior good results intact ? a failed/empty re-run never destroys prior
data.
296         """
297         if segments:
298             db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
299         else:
300             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
301
302
303     # --- API Endpoints ---
304     @app.get("/api/config")
305     def get_config():
306         # P0b: this endpoint is unauthenticated, so never serialize a credential-shaped
field. Today
307         # AppConfig holds no secrets (the Gemini key is env-only; cf_access_aud is a public
id), so this
308         # is byte-identical for the real fields ? but it fails-closed if a secret is ever
added (guarded
309         # by tests/test_redact.py + the /api/config redaction test).
310         if global_config is None:
```

```
311             return {}
312         return redact_secrets(global_config.model_dump(mode="json"))
313
314     @app.get("/api/health")
315     def health():
316         """Cheap liveness/readiness probe (no auth) for the SPA's offline banner
317         (HOSTING_PLAN.md:42). Reports the configured sport + default segmenter so the UI can
318         confirm the engine is reachable and which detector is wired. Never raises."""
319         sport = getattr(global_config, "sport", None)
320         indexing = getattr(global_config, "indexing", None)
321         return {
322             "status": "ok",
323             "sport": sport,
324             "default_segmenter": getattr(indexing, "default_segmenter", None),
325         }
326
327
328     # P1: the bundled SPA pins this ? the contract version of the /api surface. BUMP when an
existing
329     # endpoint's request/response shape changes incompatibly (purely additive endpoints need
no bump).
330     API_VERSION = 1
331
332
333     def _engine_version() -> str:
334         """Installed package version (wheel/editable); 'unknown' if not importable as a
distribution."""
335         try:
336             return _pkg_version("badminton-highlight-indexer")
337         except PackageNotFoundError:
338             return "unknown"
339
340
341     @app.get("/api/version")
342     def api_version():
343         """Version handshake so a bundled SPA can detect engine skew (PACKAGING_PLAN.md
P1)."""
344         return {"engine_version": _engine_version(), "api_version": API_VERSION, "ui":
_bundled_ui_version()}
345
346
347     @app.get("/api/capabilities")
348     def capabilities():
349         """Honest probe of what THIS box can do ? the first-run wizard renders the compute
step from
350         it (PACKAGING_PLAN.md P1). No secrets: reports only whether a Gemini key is PRESENT,
never its
351         value. Best-effort + never raises (the wizard degrades gracefully)."""
352         indexing = getattr(global_config, "indexing", None)
353         deployment = getattr(global_config, "deployment", None)
354         ffmpeg = shutil.which(ffmpeg_bin())  # P2a: honor RALLY_FFMPEG / RALLY_FFPROBE
355         ffprobe = shutil.which(ffprobe_bin())
356         # WASB weights are best-effort: env first (matches native_wasb_runner), then config.
357         wasb = getattr(indexing, "wasb", None)
358         weights_path = os.environ.get("WASB_WEIGHTS") or getattr(wasb, "weights_path", "")
or ""
359         # Cheap GPU heuristic: nvidia-smi on PATH (matches scripts/doctor.py). Deliberately
NOT
360         # torch.cuda.is_available() ? torch is absent in the LIGHT tier (so it would false-
negative on a
361         # real GPU box) and importing it would add seconds to this probe. The detector
healthcheck is the
362         # authority on whether GPU inference actually works; this is just the wizard's
first-order signal.
363         nvidia_smi = shutil.which("nvidia-smi")
```

```
        return {
            "api_version": API_VERSION,
            "engine_version": _engine_version(),
            "segmenters": sorted(segmenter_registry.list_available().keys()),
            "default_segmenter": getattr(indexing, "default_segmenter", None),
            "ffmpeg": {"ffmpeg": bool(ffmpeg), "ffprobe": bool(ffprobe),
                       "ffmpeg_path": ffmpeg, "ffprobe_path": ffprobe},
            "gpu": {"nvidia_smi_present": bool(nvidia_smi)},
            "gemini_key_present": bool(os.environ.get("GEMINI_API_KEY")),
            "wasb_weights_present": bool(weights_path) and os.path.exists(weights_path),
            "deployment": {
                "profile": getattr(deployment, "profile", "") or "",
                "compute_target": getattr(deployment, "compute_target", "") or "",
                # item 3: can this server satisfy a per-request compute="cloud" dispatch?
The SPA shows
                # the "run in the cloud" toggle only when true (else it's local-only).
                "cloud_available": _cloud_available(),
            },
        }


    @app.get("/api/videos")
    def list_videos(limit: Optional[int] = None):
        """List indexed videos (most recent first) so the console survives a reload ?
previously only a
        single-row get existed (PACKAGING_PLAN.md P1)."""
        return {"videos": db_instance.list_videos(limit=limit)}


    def _reels_dir() -> str:
        return getattr(getattr(global_config, "output", None), "output_dir", "output") or
"output"


    _REEL_EXTS = (".mp4", ".mov")


    @app.get("/api/reels")
    def list_reels():
        """List compiled highlight reels present in the output dir, newest first, with a
download URL.

        NB: reels are stored FLAT in output_dir under the bare name passed to /api/compile ?
the engine
        does NOT track a video_id?reel association today (a per-video listing would be a
phantom), so
        this is a global list. (Per-video reel tracking would need a DB record + compile
change ? a
        later slice.) Only top-level video files are listed (per-video scratch subdirs are
skipped)."""
        base = _reels_dir()
        reels = []
        try:
            entries = os.scandir(base)
        except OSError:
            return {"reels": []}
        with entries:
            for e in entries:
                if e.is_file() and e.name.lower().endswith(_REEL_EXTS):
                    try:
                        st = e.stat()
                    except OSError:
                        continue
                    reels.append({
                        "filename": e.name,
```

```
                        "size_bytes": st.st_size,
                        "modified_at": st.st_mtime,
                        "download_url": f"/api/reels/{quote(e.name)}",
                    })
        reels.sort(key=lambda r: r["modified_at"], reverse=True)
        return {"reels": reels}


    @app.get("/api/reels/{filename}")
    def download_reel(filename: str):
        """Stream a compiled reel by its bare filename (no video_id ? reels are flat in
output_dir).
        Mirrors /api/highlights' safety (bare-name + containment checks)."""
        try:
            name = safe_output_filename(filename)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        base = _reels_dir()
        path = os.path.join(base, name)
        try:
            path = resolve_under_root(path, base)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        if not os.path.isfile(path) or not name.lower().endswith(_REEL_EXTS):
            raise HTTPException(status_code=404, detail=f"No compiled reel named '{name}'.")
        media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
        return FileResponse(path, media_type=media, filename=name)


    def _ingest_remote_segments(video_id: str, outputs_uri: str, storage_config: dict,
                                *, max_rows: int = 20000) -> int:
        """Read a cloud job's ``intervals.csv`` (start,end,shot_count,ending_reason) from
the bucket and
        persist each rally via the SAME ``db.add_play_segment`` the in-process segmenters
call ? so
        ``/api/query`` + ``/api/compile`` read the rows with zero knowledge of where they
were produced.
        Returns the count ingested. Mirrors ``cloud_run._read_json``'s fetch-to-a-real-dst
pattern (cloud
        backends REQUIRE a dst). Hardened against a hostile/corrupt worker output: a
malformed row is
        skipped, non-finite / overflowing values are rejected, the loop is bounded, and the
temp dir is
        cleaned. May raise (missing/unreadable CSV, fetch error) ? the caller treats that as
a job failure
        and prunes any partial rows (destroy-AFTER-confirm)."""
        uri = f"{outputs_uri.rstrip('/')}/intervals.csv"
        ref = storage.parse_uri(uri)
        n = 0
        with tempfile.TemporaryDirectory(prefix="cr-ingest-") as td:
            local = storage.fetch(uri, os.path.join(td, ref.name or "intervals.csv"),
config=storage_config)
            with open(local, newline="", encoding="utf-8") as f:
                for row in csv.DictReader(f):
                    if n >= max_rows:
                        logger.warning("[API] cloud ingest hit the %d-row cap (%s) ?
ignoring the rest.",
                                       max_rows, uri)
                        break
                    try:
                        start, end = float(row["start"]), float(row["end"])
                    except (KeyError, TypeError, ValueError):
                        continue
                    if not (math.isfinite(start) and math.isfinite(end)):
                        continue  # reject inf/nan times rather than persist a junk rally
```

```
                    meta: Dict[str, Any] = {"source": "cloud_run",
                                            "ending_reason": (row.get("ending_reason") or
"").strip()}
                    sc = (row.get("shot_count") or "").strip()
                    if sc:
                        try:
                            meta["shot_count"] = int(float(sc))  # OverflowError on
'inf'/'1e400'
                        except (ValueError, OverflowError):
                            meta["shot_count"] = sc
                    db_instance.add_play_segment(video_id, start, end, metadata=meta)
                    n += 1
        return n


    def _cloud_available() -> bool:
        """Whether THIS server can satisfy a per-request ``compute="cloud"`` dispatch (item
3).

        True when the control plane is wired for Cloud Run: either the configured default
target is
        already ``cloud_run`` (the operator chose cloud serving), or the Cloud Run env
coordinates
        (``CLOUD_RUN_JOB`` + ``GOOGLE_CLOUD_PROJECT``) and an output bucket
(``storage.output_root``)
        are present so a forced per-request override can actually dispatch + collect a
result.

        Surfaced in ``/api/capabilities`` so the SPA only offers the cloud toggle when it
would work."""
        deployment = getattr(global_config, "deployment", None)
        if (getattr(deployment, "compute_target", "") or "") == "cloud_run":
            return True
        has_job = bool(os.environ.get("CLOUD_RUN_JOB") and
os.environ.get("GOOGLE_CLOUD_PROJECT"))
        out_root = (getattr(getattr(global_config, "storage", None), "output_root", "") or
"").strip()
        return bool(has_job and out_root)


    def _maybe_dispatch_remote(req: ProcessRequest, video_id: str, seg_name: str,
val_results,
                               play_wm: int, cand_wm: int, notify) ->
Optional[Tuple[Dict[str, Any], bool]]:
        """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
``deployment.compute_target
        == "cloud_run"``, run the heavy DETECT on a Cloud Run GPU Job and INGEST its rally
intervals into
        THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.

        Returns ``(response_dict, ran)`` when the cloud path is taken ? ``ran`` is whether
the cloud job
        actually EXECUTED (``False`` for a pre-dispatch config rejection, so the
observability report
        doesn't claim a fake segmentation run) ? or ``None`` to fall through to the in-
process segmenter
        (**DEFAULT-OFF**: ``get_compute_backend`` returns ``None`` for every
non-``cloud_run`` target, so
        the in-process path stays byte-identical)."""
        from backend.compute import ComputeJobSpec, get_compute_backend

        def _failed(msg: str, ran: bool) -> Tuple[Dict[str, Any], bool]:
            # Shape-match the in-process segmenter-fail branch + drop any partial NEW rows
(id > play_wm);
            # prior good rows (id <= play_wm) are preserved (destroy-AFTER-confirm).
```

```
            db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
            # Provenance even on failure, and HONEST about what ran: `path` is "cloud_run"
only when the
            # job actually dispatched + executed remotely (ran=True, e.g. a non-zero worker
rc / ingest
            # error); a pre-dispatch reject (ran=False) ran NOWHERE ? "not_run". `target`
records the
            # intended backend, `requested` the picker choice, `dispatched` whether the
remote job ran.
            return ({"video_id": video_id, "status": "failed", "message": msg,
                     "results": [r.model_dump() for r in val_results], "play_segments": [],
                     "compute": {"path": "cloud_run" if ran else "not_run", "requested":
req.compute,
                                 "target": "cloud_run", "dispatched": ran}}, ran)

        # SPA compute-picker (item 3): a per-request choice overrides the server's
configured default.
        # "local" forces the in-process segmenter; "cloud" forces a Cloud Run dispatch
(guarded by
        # _cloud_available so we fail clearly rather than silently running local); None /
unknown ? default.
        choice = (req.compute or "").strip().lower()
        if choice and choice not in ("local", "cloud"):
            logger.warning("[API] ignoring unknown compute=%r (expected 'local'|'cloud');
using server default.",
                           req.compute)
            choice = ""
        if choice == "local":
            return None  # explicit "run on my machine" ? in-process regardless of the
server's target
        if choice == "cloud":
            if not _cloud_available():
                return _failed("cloud-serving was requested but this server isn't configured
for it "
                               "(no Cloud Run target). Pick 'run on my machine', or
configure cloud serving.",
                               False)
            compute = get_compute_backend(global_config, target_override="cloud_run")
        else:
            compute = get_compute_backend(global_config)  # server default (default-OFF
unless cloud_run)
        if compute is None:
            return None  # default-OFF ? run the in-process segmenter below (unchanged)

        storage_cfg = getattr(global_config, "storage", None)

        # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local
to this box.
        try:
            input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
        except ValueError as e:
            return _failed(f"cloud-serving: unresolvable input '{req.video_path}': {e}",
False)
        if input_ref.backend == "local":
            return _failed("cloud-serving needs a cloud input (e.g. gs://?); a path local to
this machine "
                           "can't be read by the Cloud Run job. Put the video in your bucket
first.", False)
        out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
        if not out_root:
            return _failed("cloud-serving needs storage.output_root set to a bucket (e.g.
gs://your-bucket) "
                           "? that's where the Cloud Run job writes the result.", False)

        notify("dispatching (cloud_run)")
```

```
        logger.info("[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
req.video_path, out_root)
        # Forward the engine's AI-handoff stance so the cloud job runs the SAME detection:
with
        # skip_ai_handoff=True the WASB windows become rallies directly (no Gemini key
needed); without
        # forwarding the cloud job would default to the handoff and yield 0 rallies when it
has no key.
        # (player_pool / max_frames are still dropped on the cloud path ? a separate
documented follow-up.)
        overrides = []
        sk = getattr(getattr(global_config, "indexing", None), "skip_ai_handoff", None)
        if sk is not None:
            overrides.append(f"indexing.skip_ai_handoff={'true' if sk else 'false'}")
        spec = ComputeJobSpec(video_uri=req.video_path, out_uri=out_root,
segmenter=seg_name,
                              config_overrides=tuple(overrides))
        try:
            result = compute.run_infer(spec)  # dispatch the Cloud Run Job + bucket-poll to
completion
        except Exception as e:  # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
            logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
            return _failed(f"cloud dispatch error: {e}", True)
        if result.returncode != 0:
            return _failed(f"cloud job failed (returncode {result.returncode}).", True)
        if result.video_id and result.video_id != video_id:
            # The worker re-hashes the bucket bytes; a divergence means the input changed
between this
            # box's hash and the worker's ? ingest under the LOCAL id but flag it so it's
diagnosable.
            logger.warning("[API] cloud video_id %s != local %s ? input bytes differ between
the API hash "
                           "and the worker hash; ingesting under the local id.",
result.video_id, video_id)

        notify("ingesting (cloud_run)")
        storage_dict = storage_cfg.model_dump() if hasattr(storage_cfg, "model_dump") else
{}
        try:
            _ingest_remote_segments(video_id, result.outputs_uri, storage_dict)
        except Exception as e:  # noqa: BLE001 ? a missing/partial/corrupt result must not
500 or leak rows
            logger.error("[API] cloud ingest failed: %s", e, exc_info=True)
            return _failed(f"cloud-serving: could not read the result from
{result.outputs_uri}: {e}", True)

        # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then
drops the old ones.
        segments = [s for s in db_instance.query_play_segments(video_id, {}) if
int(s.get("id", 0)) > play_wm]
        _finalize_reingest(video_id, segments, play_wm, cand_wm)
        # Compute-path PROVENANCE: the GCP-verifiable proof the cloud path ran. `execution`
is the real
        # Cloud Run execution name (cross-check: gcloud run jobs executions describe
<execution>).
        provenance = {
            "path": "cloud_run",
            "requested": req.compute,
            "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
            "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
            "execution": result.execution,
            "outputs_uri": result.outputs_uri,
        }
        logger.info("[COMPUTE] cloud_run dispatched: execution=%s job=%s region=%s out=%s
```

```
video_id=%s",
                    result.execution, provenance["job"], provenance["region"],
result.outputs_uri, video_id)
        return ({"video_id": video_id, "status": "success",
                 "message": f"Successfully indexed {len(segments)} play segments (cloud_run
/ '{seg_name}').",
                 "results": [r.model_dump() for r in val_results], "play_segments":
segments,
                 "compute": provenance}, True)


    def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
        """The full hash ? validate ? segment ? report pipeline for one video.

        This is the former synchronous /api/process body, unchanged in behavior, now run on
        the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
        the sync endpoint used to return (UI compatibility); the per-video observability
        report is still emitted in `finally:` and grafted as response["reports"].

        `progress` is an optional callable(stage: str) used to surface coarse job progress.
        Raises on unexpected internal errors ? the caller records those as a job failure
        (after this function has already restored the pre-run segments, see below).
        """
        notify = progress or (lambda stage: None)

        notify("hashing")
        # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
        # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
        # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
        # consumers (hash / validators / segmenter) use the resolved local path.
        local_video = storage.acquire_local_input(req.video_path, getattr(global_config,
"storage", None))
        video_id = compute_video_id(local_video)
        was_reingest = db_instance.get_video(video_id) is not None

        # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
        notify("validating")
        logger.info(f"Running validations for {req.video_path}...")
        val_results, val_context = registry.run_all_with_context(local_video, global_config)
        failures = [r for r in val_results if not r.passed]

        # typed AppConfig: attribute access for the default segmenter (with safe fallback)
        default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
        seg_name = req.segmenter or default_seg
        segmenter_actual = None
        segmentation_ran = False
        # Compute-path PROVENANCE (the validation method): which backend actually ran the
DETECT ?
        # "in_process" once the local segmenter is reached, set on the response as
"cloud_run" by
        # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
fail).
        compute_actual: Optional[str] = None
        response: Optional[Dict[str, Any]] = None
        try:
            if failures:
                # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
                # status; it no longer destroys the video's prior good segments.
                db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
for r in val_results])
```

```
                response = {
                    "video_id": video_id,
                    "status": "failed",
                    "message": "Video failed validation checks.",
                    "results": [r.model_dump() for r in val_results],
                }
                return response

            # 2. Run segmenter & indexer
            logger.info("[API] Video passed checks. Segmenting and indexing...")
            db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
r in val_results])

            segmenter_cls = segmenter_registry.get(seg_name)
            if not segmenter_cls:
                # Submit-time validation already 400s unknown names; this is the defensive
                # in-worker path (e.g. config default pointing at an unregistered
segmenter).
                available = list(segmenter_registry.list_available().keys())
                response = {
                    "video_id": video_id,
                    "status": "failed",
                    "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
                    "results": [r.model_dump() for r in val_results],
                    "play_segments": [],
                }
                return response

            logger.info(f"[API] Using segmenter: {seg_name}")
            notify(f"segmenting ({seg_name})")
            segmenter_actual = seg_name
            # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
            # if the run yields segments; otherwise drop the new partial rows and keep the
old.
            play_wm, cand_wm = db_instance.segment_watermarks(video_id)
            # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run,
run the heavy
            # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
            # DEFAULT-OFF: returns None for every non-cloud target ? the in-process
segmenter below runs
            # byte-identically. (player_pool/max_frames aren't threaded to the worker
`infer` CLI yet, so
            # they're dropped on the cloud path in this slice ? a documented follow-up.)
            remote = _maybe_dispatch_remote(req, video_id, seg_name, val_results, play_wm,
cand_wm, notify)
            if remote is not None:
                response, segmentation_ran = remote  # `ran` = did the cloud job actually
execute (report)
                return response  # the cloud branch already stamped response["compute"]
(path=cloud_run)
            # In-process from here on ? record the actual path for the provenance stamp in
`finally`.
            compute_actual = "in_process"
            segmenter = segmenter_cls(db_instance, global_config)
            try:
                result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)
            except Exception:
                # A raising segmenter must not leave half-written rows: restore the pre-run
                # state (same destroy-after-confirm contract as the Failure branch), then
let
                # the job worker record the failure.
```

```
716                 db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
717                 raise
718             segmentation_ran = True
719
720             if isinstance(result, Failure):
721                 logger.error(f"[API] Segmenter failed: {result.message}")
722                 db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
723                 response = {
724                     "video_id": video_id,
725                     "status": "failed",
726                     "message": f"Segmenter '{seg_name}' failed: {result.message}",
727                     "results": [r.model_dump() for r in val_results],
728                     "play_segments": [],
729                 }
730                 return response
731
732             segments = result.value if hasattr(result, "value") else result
733             notify("finalizing")
734             _finalize_reingest(video_id, segments, play_wm, cand_wm)
735
736             response = {
737                 "video_id": video_id,
738                 "status": "success",
739                 "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
740                 "results": [r.model_dump() for r in val_results],
741                 "play_segments": segments,
742             }
743             return response
744         finally:
745             # Compute-path PROVENANCE stamp (the validation method): the cloud branch
already set
746             # response["compute"] (path=cloud_run + the Cloud Run execution); for every
other path stamp
747             # the actual path here so the job result PROVES which backend ran (in_process /
not_run) and
748             # what the picker REQUESTED ? a silent fallback (requested cloud, ran local) is
then visible.
749             if isinstance(response, dict):
750                 response.setdefault("compute", {
751                     "path": compute_actual or "not_run",
752                     "requested": getattr(req, "compute", None),
753                 })
754             prov = response.get("compute", {}) if isinstance(response, dict) else {}
755             logger.info("[COMPUTE] video_id=%s requested=%s actual=%s dispatched=%s
status=%s", video_id,
756                         getattr(req, "compute", None), prov.get("path"),
prov.get("dispatched"),
757                         (response or {}).get("status") if isinstance(response, dict) else
None)
758             # Always emit the per-video observability report (next to the video, or to
759             # report.output_dir). Never raises. Surface the paths in the response. (The
compute-path
760             # proof lives on the job result + the [COMPUTE] log, not the report ?
obsreport's record_run
761             # doesn't surface arbitrary run signals.)
762             paths = emit_indexer_report(
763                 db=db_instance, video_id=video_id, video_path=req.video_path,
config=global_config,
764                 val_results=val_results, val_context=val_context,
765                 segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
766                 was_reingest=was_reingest, segmentation_ran=segmentation_ran,
767             )
768             if paths and isinstance(response, dict):
769                 response["reports"] = paths
```

```
770
771
772     @app.post("/api/process", status_code=202)
773     def process_video(req: ProcessRequest, request: Request):
774         """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
775
776         Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
777         client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
778         runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
779         worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
780         """
781         # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the local
single-user,
782         # unchanged). When edge_auth_enabled, the Cf-Access JWT is required + verified (a
missing or
783         # spoofed identity header fails here with 401, before any work) and tags the job's
owner_id.
784         try:
785             owner_id = edge_auth.resolve_tenant(request.headers, global_config)
786         except edge_auth.EdgeAuthError as e:
787             raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
788
789         allowed_root = getattr(getattr(global_config, "security", None),
"allowed_video_root", None)
790         try:
791             validate_input_path(req.video_path, allowed_root=allowed_root)
792             # M2: resolve the input once (path or storage URI) ? raises ValueError for an
unsupported
793             # scheme, which we map to a clean 400 (an unknown scheme must never 500).
794             input_ref = storage.resolve_input_ref(req.video_path, getattr(global_config,
"storage", None))
795         except ValueError as e:
796             raise HTTPException(status_code=400, detail=str(e)) from e
797         # The local-existence fast-fail applies only to a LOCAL input, and stats the
RESOLVED local path
798         # (input_ref.key ? e.g. local://? stripped to the bare path), not the raw URI
string. A
799         # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is verified
when the worker
800         # fetches it (a fetch miss fails the job loudly). validate_input_path above still
runs for every
801         # input: its null-byte guard always applies, and a configured allowed_video_root
(hosted mode)
802         # rejects a non-local URI as out-of-root.
803         if input_ref.backend == "local" and not os.path.exists(input_ref.key):
804             raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
805         if req.segmenter and not segmenter_registry.get(req.segmenter):
806             available = list(segmenter_registry.list_available().keys())
807             raise HTTPException(
808                 status_code=400,
809                 detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}"
810             )
811
812         job_id = uuid.uuid4().hex
813         if not db_instance.create_job(job_id, req.video_path, req.segmenter,
814                                       req.player_pool, req.max_frames, owner_id=owner_id,
815                                       locale=req.locale, compute=req.compute):
816             raise HTTPException(status_code=500, detail="Failed to persist job record.")
817         _get_executor().submit(_drain_due_jobs)
818         return JSONResponse(
819             status_code=202,
820             content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
821         )
```

```
822
823
824     @app.get("/api/jobs/{job_id}/cost")
825     def get_job_cost(job_id: str):
826         """Per-job cost (COMPUTE_DECOUPLED_SERVING M8, D8). ``cost_usd`` is the source of
truth (matches
827         GCP billing); ``cost_inr`` is a display conversion at the configured FX rate.
``cost_usd`` is
828         null until billing records it (default-OFF / the placeholder pricebook ? 0)."""
829         cost = db_instance.get_job_cost(job_id)
830         if cost is None:
831             raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
832         fx = global_config.billing.fx_inr_per_usd
833         usd = cost.get("cost_usd")
834         cost["cost_inr"] = billing.to_inr(usd, fx) if usd is not None else None
835         cost["fx_inr_per_usd"] = fx
836         return cost
837
838
839     @app.get("/api/videos/{video_id}/cost")
840     def get_video_cost(video_id: str):
841         """Aggregate per-video cost across its jobs (M8): ``total_cost_usd`` +
``total_cost_inr`` + the
842         per-job rows. A video is 1:1 with an infer job, but a re-ingest can produce
several."""
843         agg = db_instance.get_video_cost(video_id)
844         fx = global_config.billing.fx_inr_per_usd
845         agg["total_cost_inr"] = billing.to_inr(agg["total_cost_usd"], fx)
846         agg["fx_inr_per_usd"] = fx
847         return agg
848
849
850     @app.get("/api/jobs/{job_id}")
851     def get_job(job_id: str):
852         """Job state: {status, progress, result, reports}. `status` = execution state
853         (queued|running|done|failed); the pipeline verdict is result["status"]."""
854         job = db_instance.get_job(job_id)
855         if not job:
856             raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
857         result = job.get("result")
858         return {
859             "job_id": job["id"],
860             "status": job["status"],
861             "progress": job.get("progress"),
862             "video_id": job.get("video_id"),
863             "error": job.get("error"),
864             "result": result,
865             "reports": (result or {}).get("reports"),
866             "created_at": job.get("created_at"),
867             "started_at": job.get("started_at"),
868             "finished_at": job.get("finished_at"),
869         }
870
871     # --- Chunked upload (B2, PR-2) ----------------------------------------------------
872     # The chunked-upload island (in-process registry + helpers + the 4 endpoints) now lives
in
873     # backend.api.uploads as a self-contained APIRouter. It resolves the live startup config
via
874     # backend.main.global_config, so behavior ? route paths, status codes, body-size limits,
875     # sequential-part logic, and abort cleanup ? is unchanged.
876     from backend.api import uploads as uploads_api
877
878     app.include_router(uploads_api.router)
879
880
```

```
    # --- Highlight download (B3, PR-2) ------------------------------------------------

    @app.get("/api/highlights/{video_id}/{filename}")
    def download_highlight(video_id: str, filename: str):
        """Stream a compiled highlight reel ? `filename` is the bare name given to
/api/compile
        (which only writes into output_dir; the browser previously got back an unreachable
        server-local path)."""
        try:
            name = safe_output_filename(filename)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        if not db_instance.get_video(video_id):
            raise HTTPException(status_code=404, detail="Indexed video record not found.")
        output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
        path = os.path.join(output_dir, name)
        try:
            path = resolve_under_root(path, output_dir)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        if not os.path.isfile(path):
            raise HTTPException(status_code=404,
                                detail=f"No compiled file named '{name}'. Compile first via
/api/compile.")
        media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
        return FileResponse(path, media_type=media, filename=name)


    @app.post("/api/query")
    def query_play_segments(req: QueryRequest):
        filters = {
            "server": req.server,
            "receiver": req.receiver,
            "duration_min": req.duration_min,
            "event_type": req.event_type
        }
        segments = db_instance.query_play_segments(req.video_id, filters)
        return {"play_segments": segments}

    class _CompileError(Exception):
        """A submit-time-validatable compile problem ? carries the HTTP status + detail so
the API path
        maps it to an HTTPException (fast 4xx) and the worker path maps it to a failed-job
result."""
        def __init__(self, status_code: int, detail: str):
            super().__init__(detail)
            self.status_code = status_code
            self.detail = detail


    def _resolve_compile(video_id: str, segment_ids: List[int], output_filename: str):
        """Shared compile resolution ? run at SUBMIT (fast-fail 4xx) AND in the WORKER (re-
resolve, since a
        row could change between submit and run). Verifies the video exists, the requested
segments match
        and survive the ``stitching.min_shot_count`` floor (segments WITHOUT a known
shot_count are exempt
        so manual/legacy picks can't silently vanish), and the output name is traversal-
safe. Raises
        ``_CompileError(404/400)``. Returns ``(video_row, kept_segments,
safe_out_name)``."""
        video = db_instance.get_video(video_id)
        if not video:
            raise _CompileError(404, "Indexed video record not found.")
```

```
        # Reject path traversal / absolute names (os.path.join would otherwise write
anywhere on disk).
        try:
            out_name = safe_output_filename(output_filename)
        except ValueError as e:
            raise _CompileError(400, str(e)) from e

        all_segments = db_instance.query_play_segments(video_id, {})
        matched = [s for s in all_segments if s["id"] in segment_ids]
        if not matched:
            raise _CompileError(400, "No matching play segments found to stitch.")

        stitching = getattr(global_config, "stitching", None) or StitchingConfig()
        kept = []
        for s in matched:
            meta = s.get("metadata") or {}
            sc = meta.get("shot_count") if isinstance(meta, dict) else None
            try:
                if sc is not None and int(sc) < stitching.min_shot_count:
                    continue
            except (TypeError, ValueError):
                pass  # unparseable count -> treat as unknown, keep
            kept.append(s)
        if len(kept) < len(matched):
            logger.info("[compile] stitching.min_shot_count=%s: dropped %d/%d segments below
the floor.",
                        stitching.min_shot_count, len(matched) - len(kept), len(matched))
        if not kept:
            raise _CompileError(400, f"All {len(matched)} matching segments fall below "

f"stitching.min_shot_count={stitching.min_shot_count}.")
        return video, kept, out_name


    @app.post("/api/compile", status_code=202)
    def compile_highlights(req: CompileRequest, request: Request):
        """Submit a reel-COMPILE job (202 + job_id); poll GET /api/jobs/{job_id} for the
reel (#329).

        Async, mirroring /api/process: the ffmpeg stitch (trim + re-encode each clip +
concat + watermark)
        can run minutes on CPU ? past the ~100 s edge/proxy timeout ? so doing it
synchronously returned a
        Cloudflare 524 while the engine kept going (the reel built, but the client saw
"failed" and never
        got the URL). The cheap checks (video exists, segments match + survive the floor,
safe filename)
        fast-fail HERE with a 4xx; the stitch runs on the worker. On `done`, the job's
`result` carries
        {filepath, download_url} ? the same shape the sync endpoint returned, now under
/api/jobs/{id}."""
        # M9 edge-auth tenant (DEFAULT-OFF ? None); a missing/spoofed identity fails 401
before queuing.
        try:
            owner_id = edge_auth.resolve_tenant(request.headers, global_config)
        except edge_auth.EdgeAuthError as e:
            raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
        try:
            video, _kept, out_name = _resolve_compile(req.video_id, req.segment_ids,
req.output_filename)
        except _CompileError as e:
            raise HTTPException(status_code=e.status_code, detail=e.detail) from e

        job_id = uuid.uuid4().hex
        # Persist the video's SOURCE ref (the NOT NULL video_path); the worker re-fetches
```

```
the row by id so
989         # the freshest filepath wins. owner_id/locale ride along for M9 multi-tenant + the
localized reel.
990         if not db_instance.create_compile_job(job_id, req.video_id, video["filepath"],
991                                               req.segment_ids, out_name, locale=req.locale,
992                                               owner_id=owner_id):
993             raise HTTPException(status_code=500, detail="Failed to persist compile job
record.")
994         _get_executor().submit(_drain_due_jobs)
995         return JSONResponse(
996             status_code=202,
997             content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
998         )
999
1000
1001    def _execute_compile(job: Dict[str, Any], progress=None) -> Dict[str, Any]:
1002        """Worker handler for a ``kind='compile'`` job (#329): re-resolve the segments,
stitch the reel,
1003        and return the {status, filepath, download_url} result the SPA polls for. Returns a
1004        ``status='failed'`` result for an EXPECTED problem (missing video / no segments / a
stitch error)
1005        so the SPA always gets a clean verdict instead of a worker-crash error."""
1006        notify = progress or (lambda stage: None)
1007        params = json.loads(job.get("params") or "{}")
1008        video_id = job.get("video_id") or ""
1009        segment_ids = params.get("segment_ids") or []
1010        locale = params.get("locale")
1011
1012        notify("resolving")
1013        try:
1014            video, kept, out_name = _resolve_compile(video_id, segment_ids,
params.get("output_filename", ""))
1015        except _CompileError as e:
1016            return {"status": "failed", "message": e.detail, "video_id": video_id}
1017
1018        stitching = getattr(global_config, "stitching", None) or StitchingConfig()
1019        output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
1020        # Localize the branding watermark to the UI locale the SPA rendered in (recorded on
the job);
1021        # unknown/absent locale keeps the configured default unchanged.
1022        localized_watermark = localize_watermark(stitching.watermark, locale)
1023        compiler = VideoCompiler(output_dir, begin_padding=stitching.begin_padding,
1024                                 end_padding=stitching.end_padding,
watermark=localized_watermark)
1025        time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
1026
1027        notify("stitching")
1028        try:
1029            # The stored `filepath` is the SOURCE ref (gs://, s3://, gdrive://, or a local
path). Resolve to
1030            # a LOCAL file the stitcher can read ? identity for a local path, fetch-to-
scratch for a URI (the
1031            # same seam the process path uses); without it, compiling a bucket-ingested
video can't open gs://.
1032            local_src = storage.acquire_local_input(video["filepath"],
getattr(global_config, "storage", None))
1033            out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
1034        except Exception as e:  # noqa: BLE001 ? a stitch/ffmpeg failure is a clean job
FAILURE, not a crash
1035            logger.error("[compile] stitching failed for video_id=%s: %s", video_id, e,
exc_info=True)
1036            return {"status": "failed", "message": f"Highlight stitching failed: {e}",
"video_id": video_id}
1037
```

```
1038        # `filepath` is server-local (not browser-reachable); also return the download URL
so the SPA needn't
```

8. assistant (2026-06-22T18:49:43.838Z)

I have most of the picture. Let me read the rest of `_execute_compile` and the chunked-upload router, plus the segmenter registry default.

9. user (2026-06-22T18:49:45.276Z)

```
1038        # `filepath` is server-local (not browser-reachable); also return the download URL
so the SPA needn't
1039        # hand-build it (P6 / PER_RALLY_SELECTION.md §4.3).
1040        download_url = f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
1041        notify("finished")
1042        return {"status": "success", "filepath": out_path, "download_url": download_url,
"video_id": video_id}
1043
1044    # --- CLI Debug Utility & Orchestration ---
1045    def run_cli_diagnose_clip(video_path: str, timestamp: float):
1046        """CLI debugging utility to examine segment properties around a timestamp."""
1047        print("=" * 60)
1048        print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
1049        print("=" * 60)
1050
1051        cfg = load_config()  # legacy wrapper still works, returns dict for now
1052
1053        # 1. Validation check diagnostics
1054        print("[DIAGNOSTIC] Running validation framework...")
1055        results = registry.run_all(video_path, cfg)
1056        for r in results:
1057            status_symbol = "[PASS]" if r.passed else "[FAIL]"
1058            print(f"  {status_symbol} {r.validator_name}: {r.message}")
1059            if r.details:
1060                print(f"         Details: {r.details}")
1061
1062        # 2. Local segment analysis around the timestamp (sampling +/- 5 seconds)
1063        print("-" * 60)
1064        print("[DIAGNOSTIC] Analyzing local motion around the timestamp...")
1065        import cv2
1066        cap = cv2.VideoCapture(video_path)
1067        if not cap.isOpened():
1068            print("[FAIL] OpenCV could not open video.")
1069            return
1070
1071        fps = cap.get(cv2.CAP_PROP_FPS)
1072        total_frames = cap.get(cv2.CAP_PROP_FRAME_COUNT)
1073
1074        start_frame = max(0, int((timestamp - 3.0) * fps))
1075        end_frame = min(int(total_frames), int((timestamp + 3.0) * fps))
1076
1077        # Measure frame-by-frame changes in sample region
1078        cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
1079        prev_gray = None
1080        motions = []
1081
1082        for _ in range(start_frame, end_frame):
1083            ret, frame = cap.read()
1084            if not ret:
1085                break
1086            gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
1087            gray = cv2.GaussianBlur(gray, (21, 21), 0)
1088
1089            if prev_gray is not None:
1090                diff = cv2.absdiff(prev_gray, gray)
```

```
1091                _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
1092                motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])
1093                motions.append(motion_ratio)
1094            prev_gray = gray
1095
1096        cap.release()
1097
1098        if motions:
1099            avg_motion = sum(motions) / len(motions)
1100            # Support both legacy dict (from wrapper) and future direct model
1101            if hasattr(cfg, "indexing"):
1102                threshold = getattr(cfg.indexing, "motion_threshold", 0.08)
1103            else:
1104                threshold = cfg.get("indexing", {}).get("motion_threshold", 0.08)  # type:
ignore[attr-defined]
1105            print(f"  Sample Frame Count: {len(motions)}")
1106            print(f"  Average motion index around timestamp: {round(avg_motion, 4)}
(threshold: {threshold})")
1107            if avg_motion > threshold:
1108                print("  Result: [ACTIVE PLAY] High motion detected. Segment classifies as
active play.")
1109            else:
1110                print("  Result: [DEAD TIME] Idle movement. Segment classified as dead
time.")
1111        else:
1112            print("  [ERROR] No visual frames were extracted in the sample range.")
1113
1114        print("=" * 60)
1115
1116
1117    def _enforce_startup_or_exit(config) -> None:
1118        """M5: run the startup checks for the server path and ``sys.exit(1)`` on a FATAL
finding BEFORE
1119        any work ? a fatal incoherent profile (e.g. cloud-serving pointed at local-only
storage) OR, for
1120        the exposed server (``include_exposure=True``, M9 hardening), a half-configured
exposure (edge
1121        auth on without the ``allowed_video_root`` jail / CF team+AUD). A strict no-op when
no
1122        ``deployment.profile`` is selected AND edge auth is off (today's behaviour). No GPU
probe here ?
1123        the server stays torch-light; the detector healthcheck is the GPU authority.
Extracted so the
1124        fail-fast wiring + the default-OFF no-op are unit-testable."""
1125        try:
1126            enforce_startup_checks(config, logger=logger, include_exposure=True)
1127        except StartupCheckError as e:
```

10. user (2026-06-22T18:49:45.672Z)

```
1       """Chunked upload (B2, PR-2) ? extracted from backend.main into its own router.
2
3       Browser uploads arrive in sequential parts ? security.max_part_mb (free-tier edge
4       proxies cap request bodies at ~100 MB ? HOSTING_PLAN §2), are assembled server-side
5       under security.upload_dir, then submitted to /api/process by the returned path.
6       Upload state is in-process by design (this is the single-box alpha): a server restart
7       aborts partial uploads and the client re-inits. Per-user partitioning of upload_dir
8       arrives with PR-3 identity scoping.
9
10      The startup config lives on ``backend.main.global_config`` (set ONLY in ``main()``).
11      ``_upload_cfg`` resolves it through the ``backend.main`` module object so it always
12      reads the live value the server set at startup (and so tests that monkeypatch
13      ``backend.main.global_config`` flow through unchanged).
14      """
```

```
    import os
    import threading
    import uuid
    from typing import Any, Dict, Optional

    from fastapi import APIRouter, HTTPException, Request
    from pydantic import BaseModel

    from backend.utils.hashing import compute_video_id  # canonical file-identity hashing
    from backend.utils.paths import safe_output_filename

    router = APIRouter()

    _uploads: Dict[str, Dict[str, Any]] = {}
    _uploads_lock = threading.Lock()


    class UploadInitRequest(BaseModel):
        filename: str
        total_size_bytes: Optional[int] = None


    class UploadCompleteRequest(BaseModel):
        total_parts: int


    def _upload_cfg():
        # Resolved lazily (import inside the function) to read the LIVE startup config that
        # main() sets on the module global, and to avoid a circular import at module load
        # (main.py imports this router; this module must not import main at import time).
        import backend.main as _main
        sec = getattr(_main.global_config, "security", None)
        upload_dir = getattr(sec, "upload_dir", None) or "output/uploads"
        max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024
        part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
        exts = [e.lower().lstrip(".") for e in (getattr(sec, "allowed_upload_extensions",
None) or ["mp4", "mov"])]
        return upload_dir, max_bytes, part_bytes, exts


    def _abort_upload(upload_id: str) -> None:
        with _uploads_lock:
            st = _uploads.pop(upload_id, None)
        if st:
            try:
                os.remove(st["tmp_path"])
            except OSError:
                pass


    @router.post("/api/upload/init")
    def upload_init(req: UploadInitRequest):
        """Start a chunked upload. Returns upload_id; PUT sequential parts next."""
        upload_dir, max_bytes, part_bytes, exts = _upload_cfg()
        try:
            name = safe_output_filename(req.filename)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        ext = os.path.splitext(name)[1].lower().lstrip(".")
        if ext not in exts:
            raise HTTPException(status_code=400,
                                detail=f"Extension '.{ext}' not allowed. Allowed:
{sorted(exts)}")
        if req.total_size_bytes is not None and req.total_size_bytes > max_bytes:
            raise HTTPException(status_code=413,
```

```
                                detail=f"File exceeds the {max_bytes // (1024 * 1024)} MB
upload limit.")
        os.makedirs(upload_dir, exist_ok=True)
        upload_id = uuid.uuid4().hex
        tmp_path = os.path.join(upload_dir, f".partial-{upload_id}")
        open(tmp_path, "wb").close()
        with _uploads_lock:
            _uploads[upload_id] = {"filename": name, "tmp_path": tmp_path, "parts": 0,
"bytes": 0}
        return {"upload_id": upload_id, "max_part_mb": part_bytes // (1024 * 1024),
                "next_part": f"/api/upload/{upload_id}/part/0"}


    @router.put("/api/upload/{upload_id}/part/{index}")
    async def upload_part(upload_id: str, index: int, request: Request):
        """Append one part (raw bytes body). Parts are strictly sequential (0,1,2,...)."""
        _, max_bytes, part_bytes, _ = _upload_cfg()
        with _uploads_lock:
            st = _uploads.get(upload_id)
        if st is None:
            raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
        if index != st["parts"]:
            raise HTTPException(status_code=409,
                                detail=f"Out-of-order part: expected {st['parts']}, got
{index} (parts are sequential)")

        received = 0
        overflow = None
        with open(st["tmp_path"], "ab") as fh:
            async for chunk in request.stream():
                received += len(chunk)
                if received > part_bytes:
                    overflow = f"Part exceeds the {part_bytes // (1024 * 1024)} MB per-part
limit."
                    break
                if st["bytes"] + received > max_bytes:
                    overflow = f"Upload exceeds the {max_bytes // (1024 * 1024)} MB total
limit."
                    break
                fh.write(chunk)
        if overflow:
            _abort_upload(upload_id)
            raise HTTPException(status_code=413, detail=overflow + " Upload aborted ? re-
init to retry.")

        with _uploads_lock:
            st["parts"] += 1
            st["bytes"] += received
        return {"upload_id": upload_id, "received_parts": st["parts"], "received_bytes":
st["bytes"]}


    @router.post("/api/upload/{upload_id}/complete")
    def upload_complete(upload_id: str, req: UploadCompleteRequest):
        """Finalize: verify part count, move into place, return {path, video_id} for
/api/process."""
        upload_dir, _, _, _ = _upload_cfg()
        with _uploads_lock:
            st = _uploads.get(upload_id)
        if st is None:
            raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
        if st["parts"] == 0 or st["parts"] != req.total_parts:
            raise HTTPException(status_code=409,
                                detail=f"Expected {req.total_parts} part(s), received
{st['parts']}.")
```

```
134         final_path = os.path.join(upload_dir, st["filename"])
135         if os.path.exists(final_path):  # never overwrite an earlier upload with the same
name
136             stem, ext = os.path.splitext(st["filename"])
137             final_path = os.path.join(upload_dir, f"{stem}-{upload_id[:8]}{ext}")
138         os.replace(st["tmp_path"], final_path)
139         with _uploads_lock:
140             _uploads.pop(upload_id, None)
141         video_id = compute_video_id(final_path)
142         return {"path": os.path.abspath(final_path), "video_id": video_id,
143                 "size_bytes": os.path.getsize(final_path),
144                 "next": "POST /api/process with this path"}
145
146
147     @router.delete("/api/upload/{upload_id}")
148     def upload_abort(upload_id: str):
149         """Abort an in-flight upload and delete its partial file."""
150         with _uploads_lock:
151             known = upload_id in _uploads
152         if not known:
153             raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
154         _abort_upload(upload_id)
155         return {"status": "aborted", "upload_id": upload_id}
156
```

11. assistant (2026-06-22T18:49:50.316Z)

Now let me verify the segmenter registry (available segmenters + default) and confirm the compile job's worker dispatch by `kind`. Let me check the segmenter registry and the job_worker handler routing.

12. user (2026-06-22T18:49:52.450Z)

```
=== segmenter registrations (register decorator/calls) ===
backend/pipeline/segmenters/fusion_hybrid.py:189:segmenter_registry.register(FusionSegmenter)
backend/pipeline/segmenters/gemini.py:570:segmenter_registry.register(GeminiPlaySegmenter)
backend/pipeline/segmenters/motion.py:255:segmenter_registry.register(MotionPlaySegmenter)
backend/pipeline/segmenters/tracknet_hybrid.py:42:segmenter_registry.register(TrackNetHybridSegm
enter)
backend/pipeline/segmenters/wasb_hybrid.py:49:segmenter_registry.register(WasbHybridSegmenter)
backend/pipeline/segmenters/yolo_hybrid.py:438:segmenter_registry.register(HybridYoloSegmenter)
```

13. user (2026-06-22T18:49:53.305Z)

```
C:\Users\avidu\Projects\bhi-computeval\backend\config\models.py:128:    default_segmenter: str =
"yolo_hybrid"
C:\Users\avidu\Projects\bhi-computeval\backend\config\models.py:210:
@field_validator("default_segmenter")
C:\Users\avidu\Projects\bhi-computeval\backend\config\loader.py:219:#   * default_segmenter
"motion"  ? pure-OpenCV, no torch / GPU / weights / API key (LIGHT tier);
C:\Users\avidu\Projects\bhi-computeval\backend\config\loader.py:227:    "indexing":
{"default_segmenter": "motion", "use_gpu": False},
```

14. assistant (2026-06-22T18:49:58.751Z)

Let me confirm the segmenter `name` attributes (the registry keys clients receive) and verify how the worker routes compile vs process jobs by `kind`.

15. user (2026-06-22T18:50:01.038Z)

```
=== segmenter name attrs ===
backend/pipeline/segmenters/base.py:43:            name = temp_inst.name
backend/pipeline/segmenters/gemini.py:81:        sport_name = self.config.get("sport",
"badminton")
```

```
backend/pipeline/segmenters/gemini.py:90:        provider_name = self.config.get("ai_provider",
idx_cfg.get("ai_provider", "gemini"))
backend/pipeline/segmenters/gemini.py:514:        model_name = self.config.get("ai_model",
self.config.get("indexing", {}).get("ai_model", "gemini-2.5-flash"))
backend/pipeline/segmenters/trajectory_hybrid.py:146:        sport_name =
self.config.get("sport", "badminton")
backend/pipeline/segmenters/trajectory_hybrid.py:160:        provider_name =
self.config.get("ai_provider", idx_cfg.get("ai_provider", "gemini"))
backend/pipeline/segmenters/trajectory_hybrid.py:162:            model_name = "local-noai"
backend/pipeline/segmenters/trajectory_hybrid.py:167:            model_name =
self.config.get("ai_model", idx_cfg.get("ai_model", "gemini-2.5-flash"))
backend/pipeline/segmenters/yolo_hybrid.py:66:        sport_name = self.config.get("sport",
"badminton")
backend/pipeline/segmenters/yolo_hybrid.py:75:        provider_name =
self.config.get("ai_provider", idx_cfg.get("ai_provider", "gemini"))
backend/pipeline/segmenters/yolo_hybrid.py:76:        model_name = self.config.get("ai_model",
idx_cfg.get("ai_model", "gemini-2.5-flash"))
backend/pipeline/segmenters/yolo_hybrid.py:377:        provider_name =
self.config.get("ai_provider", idx_cfg.get("ai_provider", "gemini"))
backend/pipeline/segmenters/yolo_hybrid.py:378:        model_name = self.config.get("ai_model",
idx_cfg.get("ai_model", "gemini-2.5-flash"))

=== list_available impl ===
39:    def register(self, segmenter_cls: Type[BasePlaySegmenter]):
48:    def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
51:    def list_available(self) -> Dict[str, str]:
```

16. user (2026-06-22T18:50:02.327Z)

```
9:every cross-function seam ? `db_instance`, `global_config`, `_execute_processing`,
46:    is the segmenter's `_execute_processing`, run UNDER the destroy-after-confirm contract.
69:        # v8 compute-picker: the worker runs _execute_processing ? _maybe_dispatch_remote,
which
88:    # Dispatch by kind: 'compile' (#329, the async reel stitch) vs 'process' (NULL/default ?
the
90:    # jobs feed the M8 cost ledger + M11 flywheel (a compile has no detection
usage/segments).
91:    is_compile = job.get("kind") == "compile"
93:        if is_compile:
94:            result = _main._execute_compile(
98:            result = _main._execute_processing(
103:        if not is_compile:
```

17. assistant (2026-06-22T18:50:05.655Z)

```
Let me confirm the segmenter `name` class attributes (the actual registry keys) and the
registry's register/list_available logic.
```

18. user (2026-06-22T18:50:07.093Z)

```
30              "no segments found" from "something went wrong" (missing key, healthcheck
31              failure, etc.). This is the start of the standardized error/result contract.
32              """
33              pass
34
35      class SegmenterRegistry:
36          def __init__(self):
37              self._segmenters: Dict[str, Type[BasePlaySegmenter]] = {}
38
39          def register(self, segmenter_cls: Type[BasePlaySegmenter]):
40              """Registers a new segmenter class under its name property."""
41              try:
42                  temp_inst = segmenter_cls(None, {})
43                  name = temp_inst.name
```

```
44              except Exception as e:
45                  raise ValueError(f"Failed to resolve name property from segmenter class
{segmenter_cls.__name__}: {e}") from e
46              self._segmenters[name] = segmenter_cls
47
48          def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
49              return self._segmenters.get(name)
50
51          def list_available(self) -> Dict[str, str]:
52              """Returns a dict of registered segmenter names and their descriptions."""
53              res = {}
54              for name, cls in self._segmenters.items():
55                  try:
56                      temp_inst = cls(None, {})
57                      desc = temp_inst.description
58                  except Exception:
59                      desc = "No description available."
60                  res[name] = desc
61              return res
62
63      segmenter_registry = SegmenterRegistry()
64
```

19. user (2026-06-22T18:50:08.525Z)

```
=== class-level name attrs in each segmenter ===
```

20. assistant (2026-06-22T18:50:13.137Z)

```
The `name` is a property, not a class attribute. Let me find the `name` property in each
segmenter to get the actual registry keys clients see.
```

21. user (2026-06-22T18:50:15.186Z)

```
=== name property return values ===
--- fusion_hybrid ---
55:    def name(self) -> str:
56-        return "fusion_hybrid"
57-
58-    @property
--- gemini ---
64:    def name(self) -> str:
65-        return "gemini"
66-
67-    @property
--- motion ---
13:    def name(self) -> str:
14-        return "motion"
15-
16-    @property
--- tracknet_hybrid ---
26:    def name(self) -> str:
27-        return "tracknet_hybrid"
28-
29-    @property
--- wasb_hybrid ---
28:    def name(self) -> str:
29-        return "wasb_hybrid"
30-
31-    @property
--- yolo_hybrid ---
50:    def name(self) -> str:
51-        return "yolo_hybrid"
52-
```

```
53-    @property
```

22. assistant (2026-06-22T18:50:21.871Z)

I now have all VERIFIED facts. Note: registration depends on which segmenter modules get imported (some import torch/heavy deps); `/api/capabilities` reports whatever is actually registered at runtime. Let me verify how segmenters get imported/registered so I can report the registry contents honestly.

23. user (2026-06-22T18:50:24.419Z)

```
=== where segmenters get imported (registration trigger) ===
1:from backend.pipeline.segmenters.base import segmenter_registry, BasePlaySegmenter
2:from backend.pipeline.segmenters.motion import MotionPlaySegmenter
3:from backend.pipeline.segmenters.gemini import GeminiPlaySegmenter
4:from backend.pipeline.segmenters.yolo_hybrid import HybridYoloSegmenter
5:from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
6:from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
7:from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
--- __init__ exists? ---
backend/pipeline/segmenters/__init__.py
=== who imports the segmenter modules ===
```

24. assistant (2026-06-22T18:51:16.357Z)

I have all the verified facts I need. Here is the report.

VERIFIED reel-creation API flow ? `bhi-computeval/backend/main.py`

Provenance of this report: local checkout on branch `feat/compute-provenance` (HEAD `ee6fba8`). Note: `/api/compile` async (#329, `8bcf6ce`) and the compute-provenance block are ALREADY on `origin/master` (`d92cc10` #341). The branch this was read on is one commit ahead with `skip_ai_handoff` forwarding. All citations are `backend/main.py` unless noted.

The chain (upload/gs:// ? process ? query ? compile ? download)

A client either (a) uploads a local file via the chunked-upload endpoints to get a server-local path, OR (b) passes a `gs://` / Drive URI directly. That path/URI feeds `/api/process` (async) ? poll `/api/jobs/{id}` until `result.play_segments` exist ? `/api/query` to filter segments ? `/api/compile` (async) ? poll the job for `result.download_url` ? GET that URL to download the reel.

---

1. `GET /api/capabilities` ? `main.py:347-381`

First-run wizard probe. No request body. Response (verified fields):
- `api_version` (int, `API_VERSION = 1`, `main.py:330`), `engine_version` (str)
- `segmenters`: `sorted(segmenter_registry.list_available().keys())` ? see registry section
- `default_segmenter`: from `indexing.default_segmenter`
- `ffmpeg`: `{ffmpeg: bool, ffprobe: bool, ffmpeg_path, ffprobe_path}`
- `gpu`: `{nvidia_smi_present: bool}` (heuristic ? `shutil.which("nvidia-smi")`, deliberately NOT `torch.cuda`)
- `gemini_key_present: bool`, `wasb_weights_present: bool`
- `deployment`: `{profile, compute_target, cloud_available}` ? `cloud_available` (`_cloud_available()`, `main.py:489-503`) gates whether the SPA offers the cloud compute toggle.

2. `POST /api/process` ? `main.py:772-821` ? **ASYNC, status_code=202** (VERIFIED)

Request body `ProcessRequest` (`main.py:262-273`):
- `video_path: str` (required ? local path OR `gs://`/Drive URI)
- `player_pool: Optional[List[str]]`, `max_frames: Optional[int]`, `segmenter: Optional[str]`,

`locale: Optional[str]`, `compute: Optional[str]` (`"local"`/`"cloud"`/None)

Fast-fail at submit (immediate 4xx, before queuing): edge-auth 401 (`:786`), bad path/unknown URI scheme ? 400 (`:795`), missing LOCAL file ? 404 (`:803-804`), unknown `segmenter` ? 400 (`:805-810`). Then creates a `queued` job and submits the drain (`:812-817`).

Response (202): `{"job_id": <hex>, "status": "queued", "poll": "/api/jobs/{job_id}"}` (`:818-821`). The slow work (MD5 hash, validate, segment) runs on the worker via `_execute_processing` (`:622-769`).

3. `GET /api/jobs/{job_id}` ? `main.py:850-869` (VERIFIED)

404 on unknown id. Returns `{job_id, status, progress, video_id, error, result, reports, created_at, started_at, finished_at}`. `status` = execution state (`queued|running|done|failed`, `:853`); the pipeline verdict is **`result["status"]`** (`success|failed`). On a successful process job, `result` carries `play_segments` (`:736-742`). (Related: `GET /api/jobs/{job_id}/cost` `:824-836`, `GET /api/videos/{video_id}/cost` `:839-847`.)

4. `POST /api/query` ? `main.py:907-916` (VERIFIED ? still SYNC, returns 200)

Request body `QueryRequest` (`main.py:275-280`): `video_id: str` (required), optional `server`, `receiver`, `duration_min: float`, `event_type`. Response: `{"play_segments": [...]}` ? each segment has `id`, `start_time`, `end_time`, `metadata`. The client picks `id`s here to feed compile.

5. `POST /api/compile` ? `main.py:967-998` ? **ASYNC, status_code=202** (VERIFIED, #329)

Request body `CompileRequest` (`main.py:282-288`): `video_id: str`, `segment_ids: List[int]`, `output_filename: str`, optional `locale: str` (localizes the watermark text + font).

Fast-fail at submit via `_resolve_compile` (`:927-964`): edge-auth 401 (`:980`); video missing ? 404; unsafe filename ? 400; no matching segments ? 400; all segments below `stitching.min_shot_count` floor ? 400. Then creates a `kind='compile'` job (`:990-992`) and submits the drain.

Response (202): `{"job_id", "status":"queued", "poll":"/api/jobs/{job_id}"}` (`:995-998`). The ffmpeg stitch runs on the worker in `_execute_compile` (`:1001-1042`); on `done`, the job `result` is `{"status":"success", "filepath", "download_url", "video_id"}` (`:1042`) where `download_url = /api/highlights/{video_id}/{out_name}` (`:1040`). The async move was specifically to kill the Cloudflare 524 from a >100s synchronous stitch (`:969-976`).

6. `GET /api/highlights/{video_id}/{filename}` ? `main.py:883-904` (VERIFIED)

Streams the compiled reel (FileResponse, `video/mp4` or `video/quicktime`). `filename` = the bare name passed to `/api/compile`. 400 on unsafe name, 404 if the video record or the compiled file is missing. (Sibling: `GET /api/reels` `:398-426` lists reels flat; `GET /api/reels/{filename}` `:429-446` downloads by bare name ? note `/api/reels` does NOT track a video_id?reel association, it's a global flat list, `:404-405`.)

Chunked upload (`backend/api/uploads.py`, router included at `main.py:876-878`)

Four endpoints (VERIFIED, `uploads.py`):
- `POST /api/upload/init` (`:64`) ? body `{filename, total_size_bytes?}`. 400 on bad name/disallowed ext (default `mp4,mov`, `:50`), 413 over `max_upload_mb` (default 4096). Returns `{upload_id, max_part_mb, next_part: "/api/upload/{id}/part/0"}`.
- `PUT /api/upload/{upload_id}/part/{index}` (`:89`) ? raw bytes body; parts STRICTLY sequential (409 on out-of-order, `:97-99`); 413 over per-part (`max_part_mb`, default 95) or total limit. Returns `{upload_id, received_parts, received_bytes}`.
- `POST /api/upload/{upload_id}/complete` (`:123`) ? body `{total_parts}`. 409 on part-count mismatch. Returns **`{path, video_id, size_bytes, next: "POST /api/process with this path"}`** ? `path` is the absolute server path you feed into `/api/process`.
- `DELETE /api/upload/{upload_id}` (`:147`) ? abort + delete partial.

Upload state is **in-process / in-memory** (`_uploads` dict, `uploads.py:28`) ? a server restart aborts partial uploads by design (single-box alpha, `uploads.py:6-8`).

Available segmenters (registry / default)

`segmenter_registry` (`backend/pipeline/segmenters/base.py:35-63`). Registration is triggered by importing the modules in `backend/pipeline/segmenters/__init__.py` (all six imported there). Registry keys = each class's `name` property (VERIFIED values):
- `motion` (`motion.py:14`, pure-OpenCV LIGHT-tier), `gemini` (`gemini.py:65`), `yolo_hybrid` (`yolo_hybrid.py:51`), `tracknet_hybrid` (`tracknet_hybrid.py:27`), `wasb_hybrid` (`wasb_hybrid.py:29`), `fusion_hybrid` (`fusion_hybrid.py:56`).

`/api/capabilities` returns `sorted(...keys())` of whatever registered at runtime ? modules whose heavy deps (torch/weights) fail to import won't appear, so the served list reflects actual box capability.

Default segmenter ? two distinct values (VERIFIED, important nuance):
- Pydantic model default: `default_segmenter = "yolo_hybrid"` (`backend/config/models.py:128`).
- BUT the LIGHT / batteries-included first-run seeded config writes `default_segmenter: "motion"` (`backend/config/loader.py:227`) ? the torch-free default an installed app actually gets.

The effective default in `_execute_processing` falls back to `"motion"` if `indexing` is unset (`main.py:652`). A per-request `segmenter` overrides it (`main.py:653`).

Job `result['compute']` provenance block (VERIFIED, the compute-provenance work)

Every process job result carries a `compute` block proving which backend ran the DETECT:
- **In-process default path**: stamped in the `finally` of `_execute_processing` (`main.py:744-753`): `{"path": "in_process", "requested": req.compute}`. `path` is `"not_run"` if nothing ran (validation/config fail).
- **Cloud path** (`_maybe_dispatch_remote`, `main.py:506-619`): on success returns `{"path":"cloud_run", "requested", "job", "region", "execution", "outputs_uri"}` (`:606-613`) ? `execution` is the real Cloud Run execution name (cross-checkable via `gcloud run jobs executions describe`). On failure: `{"path": "cloud_run" if ran else "not_run", "requested", "target":"cloud_run", "dispatched": ran}` (`:527-530`) ? `dispatched` honestly distinguishes a remote run that failed from a pre-dispatch config reject. A `[COMPUTE]` log line is also emitted (`:614`, `:755-757`).

The `compute` picker: per-request `compute="local"` forces in-process even on a cloud server (`:540-541`); `"cloud"` forces dispatch but `_failed`s with 400-ish job result if `_cloud_available()` is false (`:542-546`). Default-OFF: `get_compute_backend` returns `None` for any non-`cloud_run` target, so the in-process path is byte-identical (`:550-551`).

NOT implemented / honest caveats

- **`/api/compile` is NOT synchronous** anymore ? it returns 202 + job_id; a client still expecting a synchronous reel response is broken (#329).
- **`/api/process` does NOT return segments inline** ? only `{job_id}`; segments arrive via the polled job result.
- **Cloud path drops `player_pool` / `max_frames`** ? not threaded to the worker `infer` CLI yet (documented follow-up, `main.py:573`, `:701-702`).
- **`/api/reels` has no per-video association** ? reels are stored flat under the bare compile filename; there is no video_id?reel DB record (`main.py:404-405`).
- **Upload state is non-durable** (in-memory) ? partial uploads die on restart, and per-user partitioning of `upload_dir` is deferred to PR-3 (`uploads.py:6-8`).